Waseet — Software Requirements Specification

Oct 20, 2026 · @Osama Riyad

Waseet is an AI-mediated escrow service: a buyer funds a deal through PayPal, the platform holds the money, and an AI verifies the seller's deliverable against agreed criteria to auto-release payment — or routes to a human-reviewed dispute. This SRS specifies a PayPal-sandbox prototype one person can build and demo in about three weeks for the PayPal AI Hackathon (deadline Nov 12, 2026).

Goals & success criteria

Remove the slow, expensive human escrow middleman for peer-to-peer and freelance deals, replacing it with an explainable AI verifier running on PayPal rails.

The build is successful when both the happy path (deliverable passes, funds auto-release) and the dispute path (deliverable fails, human resolves) work end to end in the PayPal sandbox, and the whole story is demonstrable in under three minutes. Deliverables required by the hackathon: a public open-source repo judges can run, and a demo video under three minutes.

How Waseet is designed to score against the five judging criteria:

Judging criterion	How Waseet addresses it
Technological implementation	The AI makes a real money-movement decision against real PayPal sandbox APIs (Orders, Payouts, Refunds, Webhooks); non-trivial state machine and audit trail.
Design	A two-sided buyer/seller experience with a deal timeline, verification report, and dispute screen — a coherent product, not a script.
Potential impact	Trust is the blocker in P2P and freelance deals; automated, explainable escrow is a credible, specific solution with a clear audience.
Innovation	Replaces the human arbiter with an explainable AI verifier plus dispute mediator; differentiated from crypto-rail escrow by being PayPal-fiat with human-in-the-loop.

Judging criterion	How Waseet addresses it
Presentation	Both the happy path and the dispute path fit inside the three-minute video, each ending in a visible balance change and audit trail.

Users & personas

Persona	Goal	Key actions
Buyer	Pay only when the agreed work is actually delivered	Agrees terms, confirms acceptance criteria, funds escrow, contests a delivery
Seller	Get paid promptly once the work is done, without chasing	Agrees terms, submits the deliverable, receives the payout
Reviewer (human-in-the-loop)	Resolve disputes the AI is not confident about	Reads the AI recommendation, decides release / refund / split
Verifier agent (system actor)	Decide whether a deliverable meets the criteria	Drafts criteria, verifies the deliverable, recommends a decision

Functional requirements

Priority: Must = required for the demo; Should = strengthens the submission; Could = stretch if time allows.

ID	Requirement	Priority
FR-1	A user can create a deal from a plain-language description (parties, amount, currency, description).	Must
FR-2	The AI drafts a structured acceptance checklist from the description; both parties can review it before funding.	Must
FR-3	The buyer funds the deal via PayPal; captured funds are held by the platform account (state = FUNDED).	Must

ID	Requirement	Priority
FR-4	The seller submits a deliverable (URL, uploaded file, image, repo link, or text).	Must
FR-5	On submission, the AI verifies the deliverable against each criterion and returns pass/fail, an overall verdict, a confidence score, and reasoning.	Must
FR-6	A high-confidence pass auto-releases funds to the seller via PayPal Payout.	Must
FR-7	A high-confidence fail, a low-confidence verdict, or a buyer contest moves the deal to DISPUTED.	Must
FR-8	In DISPUTED, the AI produces a reasoned recommendation (release / refund / split) and a human makes the final decision.	Must
FR-9	A refund decision returns the captured funds to the buyer; a split pays part to the seller and refunds the rest.	Must / Could (split)
FR-10	Every state change and decision is written to an append-only audit log linked to the PayPal reference id.	Must
FR-11	Both parties see a deal timeline showing the current state, the verification report, and any dispute outcome.	Should
FR-12	Parties are notified on key events (funded, delivered, verified, released, refunded, disputed) via webhook-driven updates.	Should
FR-13	The platform can take a configurable fee on release.	Could

Escrow lifecycle

States: DRAFT, FUNDED, DELIVERED, VERIFYING, RELEASED (terminal), REFUNDED (terminal), DISPUTED, SPLIT (terminal). RELEASED, REFUNDED and SPLIT are the only end states; every other state has at least one exit.

Current state	Event / trigger	Next state	Action
—	Deal created, criteria confirmed	DRAFT	Persist deal and acceptance criteria
DRAFT	Buyer completes PayPal checkout	FUNDED	Capture payment into platform account
FUNDED	Seller submits deliverable	DELIVERED	Store deliverable reference
DELIVERED	Verification starts	VERIFYING	Run AI verifier against criteria
VERIFYING	High-confidence pass	RELEASED	Payout to seller; log reasoning
VERIFYING	High-confidence fail or low confidence	DISPUTED	Hold funds; surface AI recommendation
FUNDED or DELIVERED	Buyer contests	DISPUTED	Hold funds; open dispute
DISPUTED	Human decides release	RELEASED	Payout to seller
DISPUTED	Human decides refund	REFUNDED	Refund captured payment to buyer
DISPUTED	Human decides split	SPLIT	Partial payout to seller, refund remainder

PayPal integration (sandbox)

Everything runs in the PayPal sandbox — no real money. Three sandbox accounts model the parties:

- **Buyer** — personal account that funds the deal.
- **Seller** — personal account that receives the payout.
- **Platform** — business account that holds funds in escrow and initiates payouts/refunds.

Lifecycle step	PayPal API	What happens
Fund escrow	Orders v2 (create + capture)	Buyer checks out; payment is captured into the platform account — "funds in escrow".

Lifecycle step	PayPal API	What happens
Release	Payouts	Platform sends the amount (optionally minus a fee) to the seller.
Refund	Payments v2 (refund capture)	Platform returns the captured payment to the buyer.
Split	Payouts + Payments v2 refund	Part is paid out to the seller; the remainder is refunded to the buyer.
Event confirmation	Webhooks	Capture, payout and refund events confirm state transitions and feed the audit log.

Optionally, the AI calls these as tools through the PayPal Agent Toolkit / MCP server (`@paypal/agent-toolkit`); otherwise the PayPal REST API is wrapped as tools directly. Funds are never released without a recorded AI or human decision — no silent auto-capture-and-release.

AI components

Component	Input	Output	Approach
Criteria drafter	Plain-language deal description	Structured acceptance checklist (list of testable criteria)	LLM with a structured-output schema; both parties confirm before funding
Deliverable verifier	Submitted deliverable + acceptance criteria	Per-criterion pass/fail, overall verdict, confidence score, reasoning	Multimodal LLM reads text/image/URL/file against each criterion
Decision policy	Verifier output + buyer contest flag	One of: auto-release, dispute	Deterministic rules over verdict + confidence; low confidence never auto-releases
Dispute mediator	Deal, deliverable, verification report	Recommendation (release / refund / split) + reasoning for a human	LLM reasoning surfaced to the reviewer; human makes the final call

The AI verdict is advisory and always backed by a dispute path; the UI states this plainly and never claims the verdict is tamper-proof.

Data model

Entity	Key fields	Notes
Deal	id, buyer, seller, description, amount, currency, state, paypal_order_id, paypal_capture_id, payout_batch_id, created_at	The central record; state follows the lifecycle table
AcceptanceCriterion	id, deal_id, text, confirmed_by_buyer, confirmed_by_seller	One row per testable criterion
Deliverable	id, deal_id, type (url/file/image/repo/text), content_ref, submitted_at	What the seller submits
VerificationReport	id, deal_id, per_criterion[], verdict, confidence, reasoning, decided_by (ai/human), decided_at	Produced by the verifier or the human reviewer
AuditLogEntry	id, deal_id, from_state, to_state, actor, reason, paypal_reference, created_at	Append-only; one row per transition or decision

Non-functional requirements

Category	Requirement
Security	PayPal credentials in environment variables, never committed; webhook signatures verified before acting on an event.
Explainability	Every release, refund or split is backed by stored, human-readable reasoning and a confidence score.
Reliability	State transitions and payouts are idempotent; a replayed webhook never double-pays or double-refunds.
Latency	Verification returns within a few seconds so the demo stays live; long model calls show a VERIFYING state, not a frozen screen.

Category	Requirement
Auditability	The audit log is append-only and each entry links to its PayPal reference id.
Privacy	Only the data needed to verify a deliverable is sent to the model; uploaded files are access-controlled to the deal's parties.
Honesty	The UI presents the AI verdict as advisory with a dispute path, not as a guarantee.

Assumptions & out of scope

Assumptions:

- All money movement is PayPal sandbox; no real funds and no production KYC.
- Both parties already have accounts in the prototype; onboarding is minimal.
- A single human reviewer resolves disputes; no multi-party arbitration.
- Deliverables are small enough for a model to inspect directly (a file, image, link, or text), not large codebases.

Out of scope for the hackathon build:

- Real custody of funds, interest on held balances, or multi-currency FX.
- Production fraud, AML, or identity verification.
- Fully autonomous dispute resolution without any human in the loop.
- Mobile apps; the prototype is web only.

Acceptance & demo scenarios

The prototype is accepted when these scenarios pass; together they are the under-three-minute demo.

Scenario	Steps	Expected result
Setup	Buyer and seller agree a deal in plain language	AI drafts acceptance criteria; both confirm
Fund	Buyer completes PayPal sandbox checkout	Deal = FUNDED; funds captured into platform account

Scenario	Steps	Expected result
Happy path	Seller submits a deliverable that meets the criteria	AI passes it with reasoning; auto-release; Payout to seller; seller sandbox balance rises
Dispute path	On a second deal, seller's deliverable misses a criterion	AI flags the failure; deal = DISPUTED; human reviews recommendation; refund to buyer
Audit	Open the deal timeline after both deals	Every state change, the AI reasoning, and the PayPal reference ids are visible

PayPal building blocks & references

The confirmed PayPal pieces Waseet builds on, from the hackathon's official resources. Use the **JavaScript SDK v6** (not the older v5) — it is PayPal's current standard and a prior hackathon team won partly for being on it.

Capability	Resource	Use in Waseet
Checkout / funding	PayPal Checkout, JS SDK v6	Buyer funds the deal
Core API	PayPal REST API	Orders capture, refunds
Release	Payouts	Pay the seller on release
Event confirmation	Webhooks	Drive state transitions, feed the audit log
Disputes (optional)	Disputes	Mirror real dispute objects if time allows
AI calls PayPal	PayPal AI Toolkit, Agent Toolkit quickstart, MCP server quickstart	Let the AI invoke PayPal as structured tools
Sandbox setup	Getting Started with PayPal sandbox	Fastest route to the three sandbox accounts

Sponsor integrations & prize lanes

Waseet answers three of PayPal's own inspiration themes directly — AI Agents / Automation ("pays contractors automatically when a job is marked complete"), Business Solutions ("splits payment automatically between a platform and its sellers"), and Money Movement ("pays out the moment a task is approved") — and the submission write-up should anchor to them explicitly.

Sponsor tools, ranked by effort-to-reward for Waseet. Plan: build core Waseet, then add the top three for extra prize lanes at little extra cost.

Sponsor	Prize lane	How Waseet uses it	Effort
AG Grid	$5k + $2k + $1k×3	Audit log and deals list as a data grid — the admin/audit view	Low
Render	Render credits	Host the live demo (needed anyway)	Low
Postman	$1k×3	Publish a Postman collection of the PayPal + agent flow	Low
APIMatic	$1k×3 + subscription	Context Plugin feeds the agent PayPal API context	Low–Med
Bryntum	$1k×3	Scheduling/timeline components for the deal-timeline hero screen	Med
Elastic	$1.5k	Only if added: "find similar past disputes" semantic search	Med
Zapier	included	Only if added: fire external notifications on deal events	Med
Kernel / Astropods / Channel3	various	Skip — no honest fit for escrow	—

Recommended add-ons: AG Grid (audit dashboard) + Render (hosting) + Postman (collection) — three lanes for roughly a day of extra work, none of it filler.